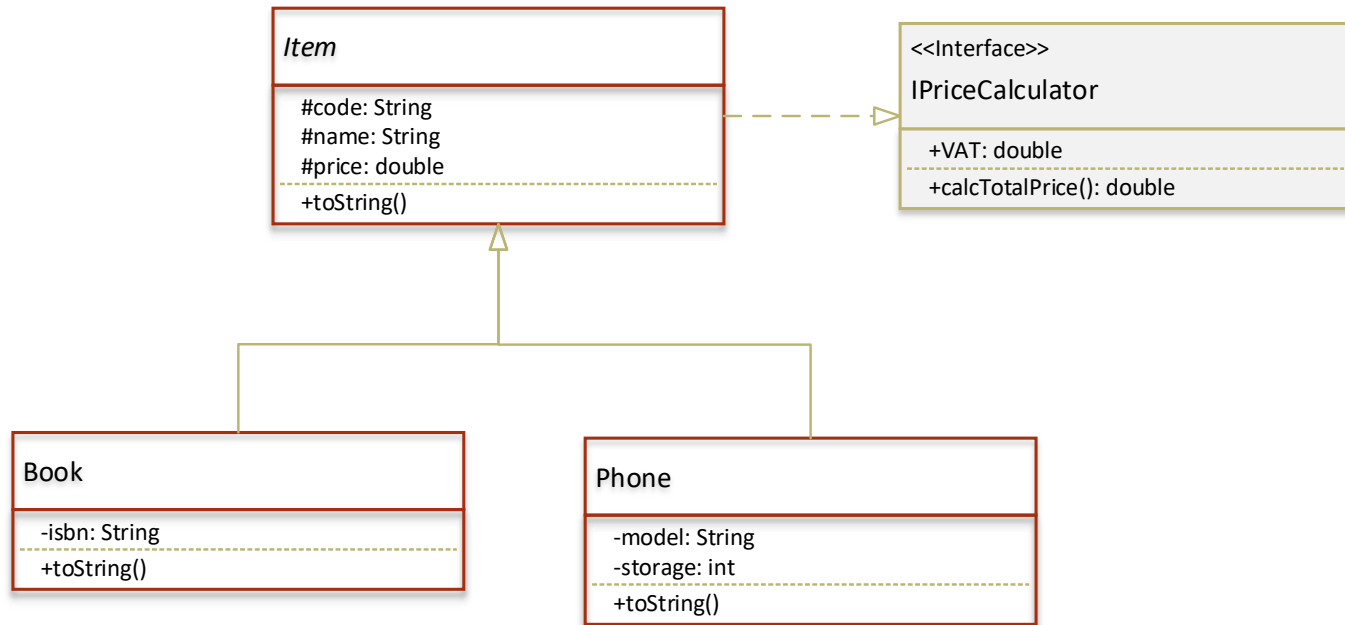


Topic 1: Inheritance, Interfaces and Design patterns

Part 3: Design patterns

A simple exercise



- **Question:** is there any difference if we removed the interface from the UML diagram and included the *abstract* methods in **Item**?

Agenda

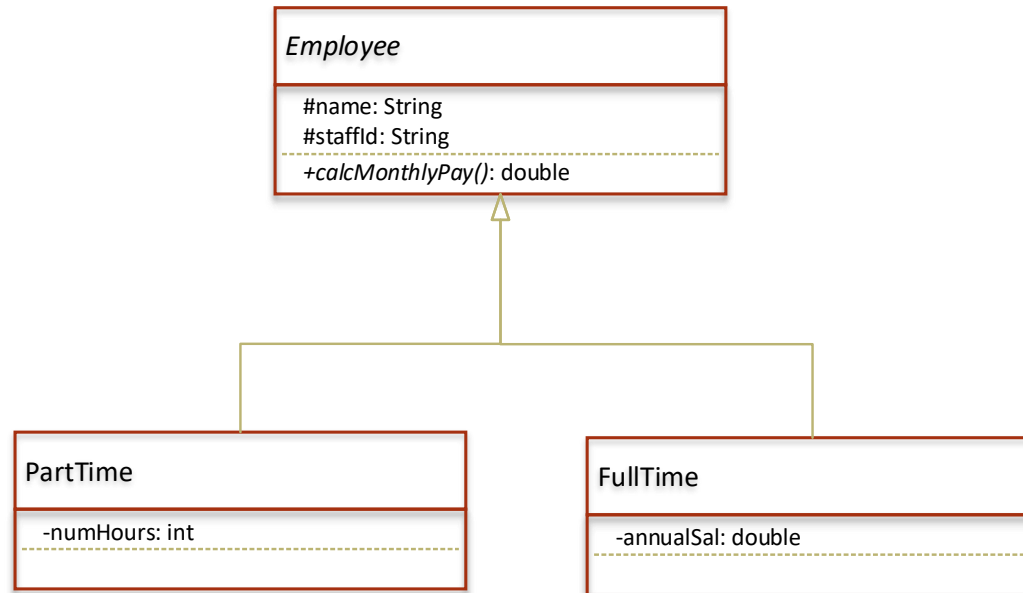
1. Overview of the previous lectures
2. Previous homework
3. Introduction
4. What is a Design pattern?
5. Why use Design patterns?
6. What are the main types of Design patterns?
7. Which Design patterns to use?
8. Using Design patterns: Examples
 1. Singleton
 2. Factory
 3. Template
 4. DAO
9. Summary
10. Questions and comments

1. Overview of previous lectures

1. Main OOP elements
2. Inheritance and polymorphism
3. Multiple inheritance
4. Abstract classes and Interfaces: which to use?
5. We have discussed
 - POCO (or POJO): Plain Old Class Object (or Plain Old Java Object)
 - Most of our programming is based on POCO (or POJO)

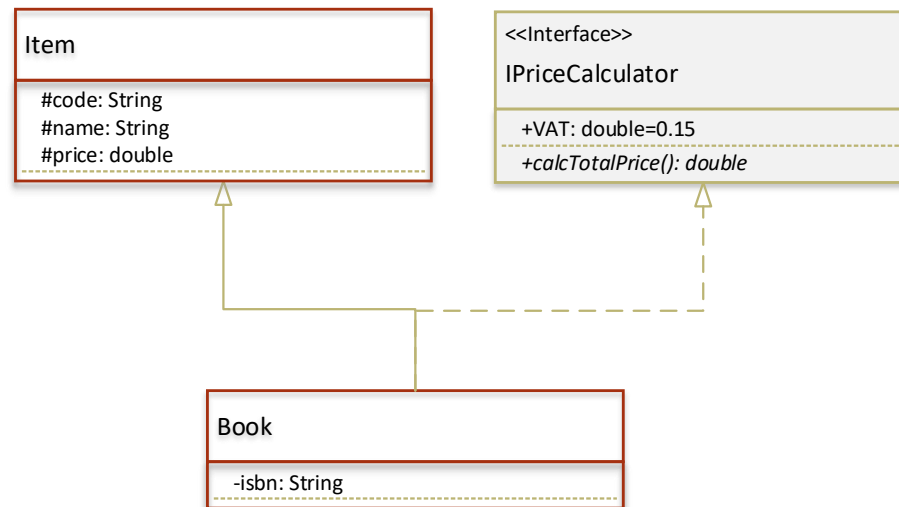
2. Previous Exercise: Inheritance/Abstract classes

1. Write the code for **Employee**, **PartTime** and **Tester** classes



2. Previous exercise: Interfaces

- Refactor and convert the UML diagram to code. Write a **Tester** to test **Item**, **PriceCalculator** and **Book**
- Questions
 1. Why is can't Item be an interface?
 2. Should Item class be abstract or not? Why?



3. Introduction

- Question: explain why the class **Lecturer** is poorly designed?

```
Lecturer.java
└─ Lecturer
    ├── lid
    ├── name
    ├── Lecturer()
    ├── Lecturer(String, String)
    ├── delete(Lecturer) : boolean
    ├── getLid() : String
    ├── getName() : String
    ├── insert(Lecturer) : boolean
    ├── search(String) : Lecturer
    ├── setLid(String) : void
    ├── setLName(String) : void
    ├── toString() : String
    └── update(Lecturer) : boolean
TestLecturer.java
└─ TestLecturer
    └── main(String[]) : void
```

- Answer: **NOT POCO**. Some methods are not part of a Lecturer but rather **act on the objects**

3. Introduction

- ❑ There are some common problems in software development, e.g., how to:
 1. Create classes and instances
 2. Design interaction (e.g., inheritance) among objects
 3. Increase cohesion and reduce coupling
- ❑ Over time, standard **approaches** or “patterns” to solving these problems have emerged
- ❑ These are based on two basic Object-oriented programming principles
 1. **Program to an interface (rather than concrete class); i.e.:**
 - ✓ Wherever possible use interface, abstract class or super class, **DON'T JUST create classes! Why?**
 2. **Composition over inheritance**
 - ✓ **Why?**
 - ✓ **We WILL NOT COVER but USE EXAMPLES of composition**

4. What is design pattern?

A **common approach** to solving recurring problems:

1. A template (not solution)
2. Tried and tested
3. Can be customised to suit your situation
4. It is about **design** not implementation
5. Independent of language
6. Communication among relevant stakeholders (e.g., analysts and developers)
7. Etc.

5. Why use design patterns?

A common approach to solving problems:

1. Standard terminology for communication
2. A way to think about designing a solution or implementation (e.g., you think in objects)
3. Best practices: you learn to develop code that is reusable, maintainable and extensible
4. A way to quickly learn software development

❑ Main problems

1. Sometimes there may be a simpler approach and no need for a design pattern
 2. Too many design patterns
- ✓ Documented: 27 formal patterns
 - ✓ Many more are informal (not documented)
 - E.g., organizations can develop their own patterns

6. What are the main types of design patterns

Design patterns are generally divided into four:

1. **Creational**: the goal is to make it efficient and flexible to **create objects**.
 - ✓ Examples: **Singleton** and **Factory** patterns
 - Remember the following:
 - a. `Person p = new Student();`
 - b. `Employee e=new PartTime();`
2. **Structural**: the goal is to make it efficient and flexible to **create structures** (classes, inheritance, interfaces, etc.)
 - ✓ Examples: **Adapter pattern** (what is it used for?)
3. **Behavioural**: the goal is efficient and flexible **interaction** among OOP structures (classes, interfaces, inheritance, etc.)
 - ✓ Examples: **Template method pattern**
 - Remember: interaction and behaviour are about **methods**
4. **Miscellaneous**: various other patterns
 - ✓ Examples: **DAO** and **MVC** design patterns

7. Which Design pattern to use?

1. It depends on the problem being solved
2. Keep it simple: use the simplest pattern for a specific situation
3. DON'T USE a pattern just because you know about it!

8. Using Design patterns: **Singleton** pattern

1. **Singleton** pattern: how to create a single (**only one**) object (i.e. instance) of a class
2. **Factory** pattern: how to create **super** and **subclasses** and use them
3. **Template** pattern: specifies the **skeleton of an algorithm** in:
 1. A superclass for subclasses to override or
 2. An interface for implementation classes to implement
 - ✓ The structure of the algorithm CANNOT changed in the superclass or the implementation class
4. **Data Access Object** pattern: separates the **data object** (i.e., business object- POCO) from the
 1. **data access processes** (i.e., business operations as interfaces) and
 2. implementation of the **business operations** (i.e. a concrete class)
5. **Adapter** pattern: how to join different interfaces

8. Using Design patterns: **Singleton** pattern

- ❑ Allows **creation of one instance** of a class
 1. Used in cases where you need to create ONLY one instance of a class)
 2. You can re-use the object you created over and over again
- ❑ Allows a safer global access (e.g., to control access to global resources)
 - Question: why is **Singleton** useful for programs which handle:
 1. Database connection
 2. Socket connection
- ❑ Practical applications
 1. A small business can have only one manager
 2. A department has only one printer
 3. A class can have only one class rep
 4. Any other?
- ❑ There are several ways to implement Singleton

Using Design Patterns: Singleton pattern

- ❑ We'll use the Lazy initialization approach
 - Common steps to write the class
 1. Create a private constructor (this means you cannot create objects of the class)
 2. Create a method (a getInstance() method) which behaves like a constructor. This method
 1. Checks: if no object has been created, then creates one
 2. If an object exists, it returns the same object every time
 3. Why static method?

Type text here

Using Design Patterns: **Singleton** pattern

1. Create a singleton class **Printer**
2. Create a class **Tester** to test **Printer**

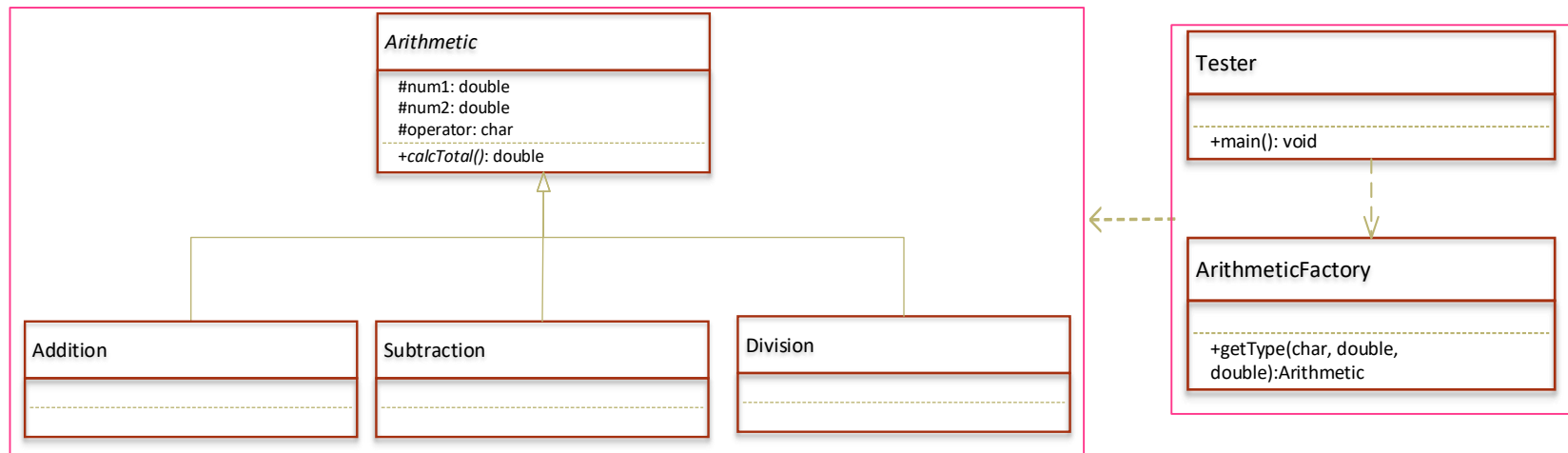
See the Singleton pattern code

```
Printer.java
└─ Printer
    └─ p
    └─ getInstance() : Printer
    └─ Printer()

Tester.java
└─ Tester
    └─ main(String[]) : void
```

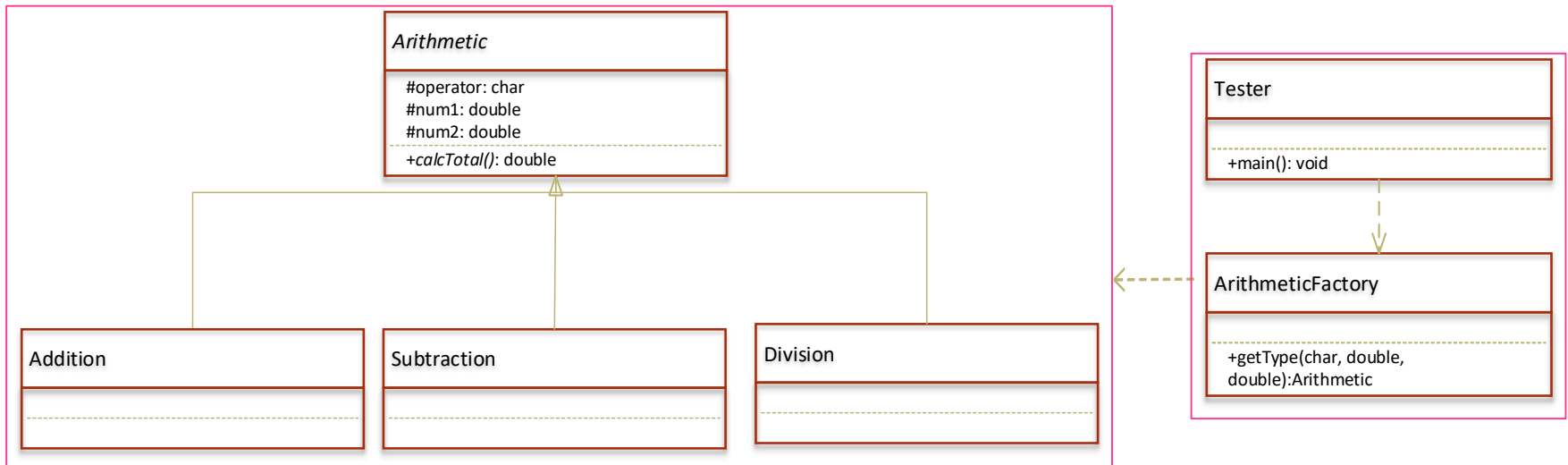
Using Design Patterns: Factory pattern

- ❑ With the factory pattern, we create a “factory” class to **manufacture** instances of the **subclasses** as instances of the **superclass**
- ❑ In the example below, **ArithmeticFactory** contains a method *calcTotal* which creates instance of:
 - ❑ Sum as Arithmetic: if the operator is ‘+’
 - ❑ Subtract as Arithmetic: if the operator is ‘-’ and
 - ❑ Division as Arithmetic, if the operator is ‘/’



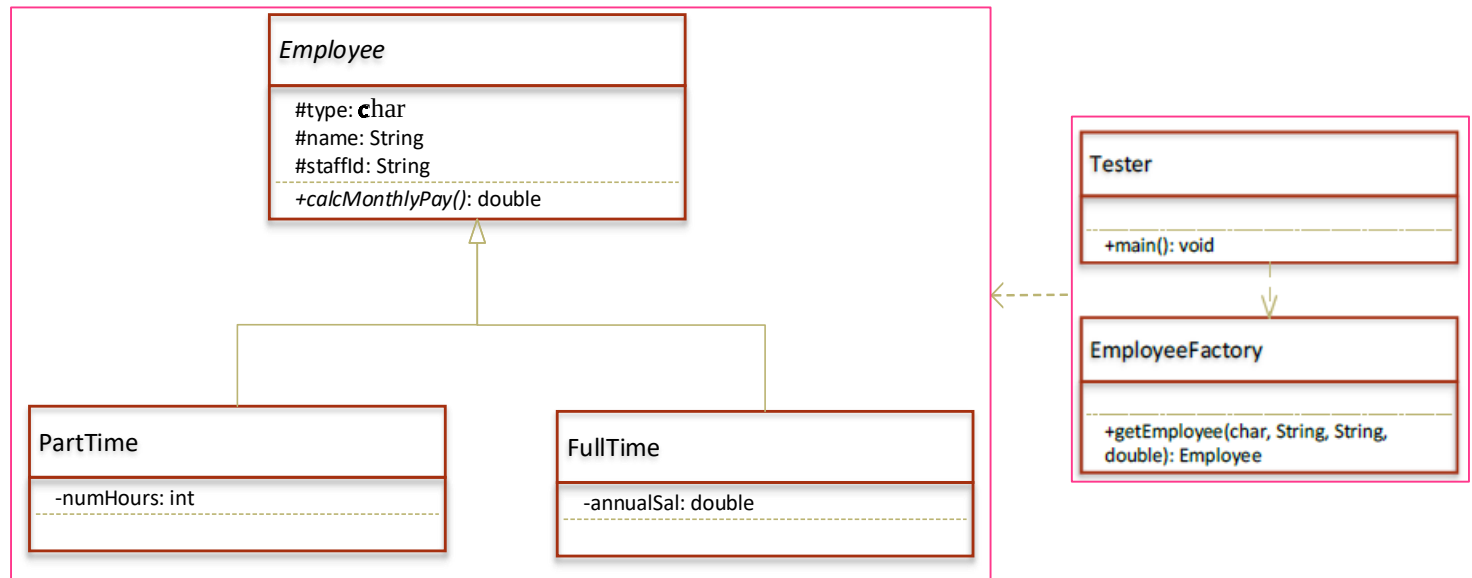
Using Design Patterns: Factory pattern

1. Create class **Arithmetic**
2. Create class **Addition** (overrides *calcTotal* to return num1 + num2)
3. Create class **Division** (overrides *calcTotal* to return num1- num2)
4. Create class **Subtraction** (overrides *calcTotal* to return num1/num2)
5. Create the **ArithmeticFactory** (factory class) and implement *getType*



Using Design Patterns: Factory pattern

- ❑ With the factory pattern, we create a “factory” class to “manufacture” instances of **subclasses** as instances of **superclass**
- ❑ In the example below, **EmployeeFactory** has a method *getEmployee* which creates and returns **instances** of:
 - PartTime as Employee, if type is ‘P’ or ‘p’
 - FullTime as Employee, if type is ; ‘F’ or ‘f’



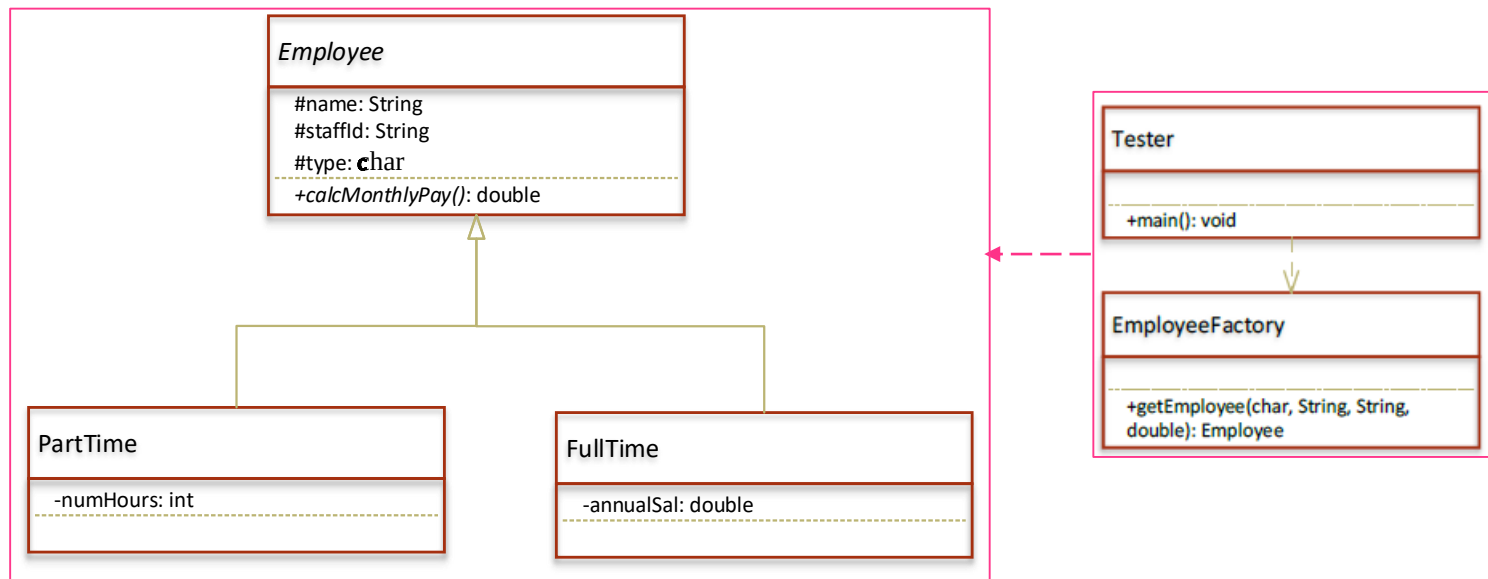
- ❑ **Refer to exercises**

Using Design Patterns: Factory patterns

❑ How to write the code

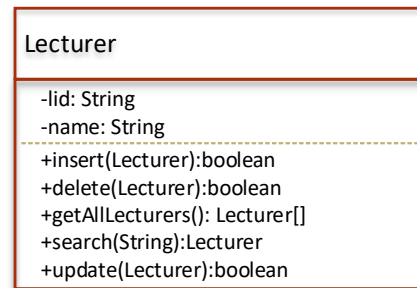
1. Create class **Employee**
2. Create class **PartTime** (override calcMonthlyPay as **numHours * 80**)
3. Create class **FullTime** (override calcMonthlyPay as **annualSal * 1/12**)
4. Create class **EmployeeFactory** (the factory class) and implement **getEmployee()**; test **type** and return PartTime or FullTime as Employee

NOTE: Refer to the exercise



Example: DAO

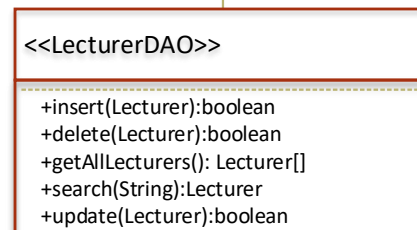
Illustration: **Lecturer** and **LecturerDAO**



Lecturer class with access methods (poor design)



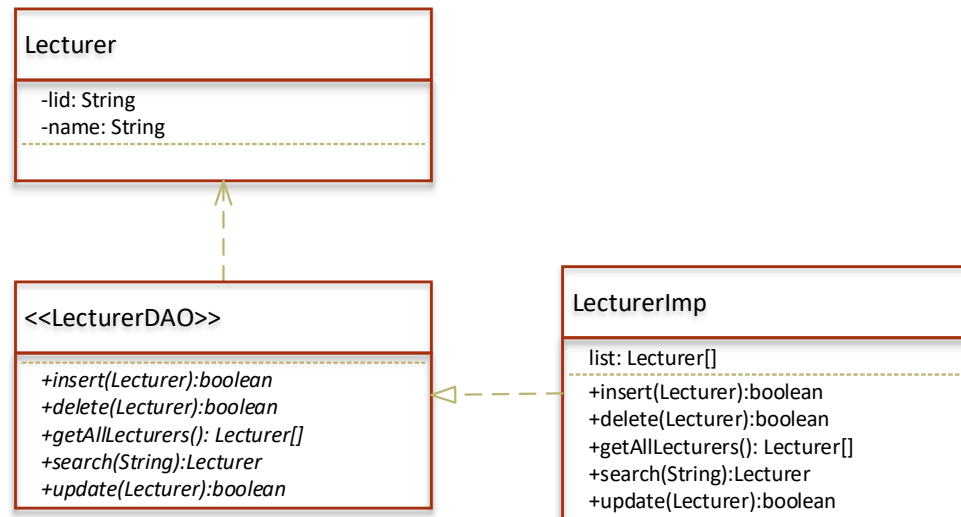
Lecturer class separated from access methods (good design).



We'll need another class to implement the interface

Example: DAO

Illustration: the Lecturer, LecturerDAO, LecturerImp, Test ([Exercise- DAO pattern](#))



Lecturer class separated from access methods (in an interface) and the implementation class

Example: DAO

Illustration: the Lecturer, LecturerDAO, LecturerImp, Test (Exercise- DAO pattern)

Lecturer

- lid
- name
- Lecturer()
- Lecturer(String, String)
- getLid() : String
- getName() : String
- setLid(String) : void
- setName(String) : void
- toString() : String

LecturerDAO

- delete(Lecturer) : boolean
- getAllLecturers() : ArrayList<Lecturer>
- insert(Lecturer) : boolean
- search(String) : Lecturer
- update(Lecturer) : boolean

LecturerImp

- list
- LecturerImp()
- LecturerImp(ArrayList<Lecturer>)
- delete(Lecturer) : boolean
- getAllLecturers() : ArrayList<Lecturer>
- insert(Lecturer) : boolean
- search(String) : Lecturer
- update(Lecturer) : boolean

Tester

- main(String[]) : void

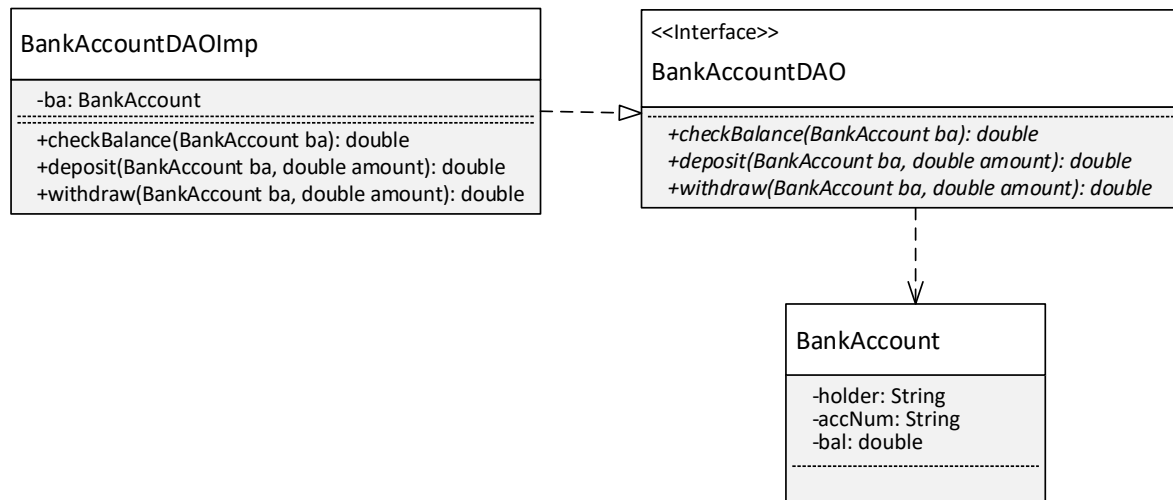
Lecturer class separated from access methods (with the implementation class and Tester)

Example: DAO

- ❑ The Lecturer, LecturerDAO, LecturerImp and Testers
- ❑ Using the Lecturer-LecturerDAO example, explain how the DAO design pattern can:
 1. Increase cohesion and reduce coupling
 2. Improve code reuse
 3. Improve code maintenance
 4. Reduce development cost

Example: DAO

- ❑ The **BankAccount**, **BankAccountDAO** and **BankAccountDAOImp**

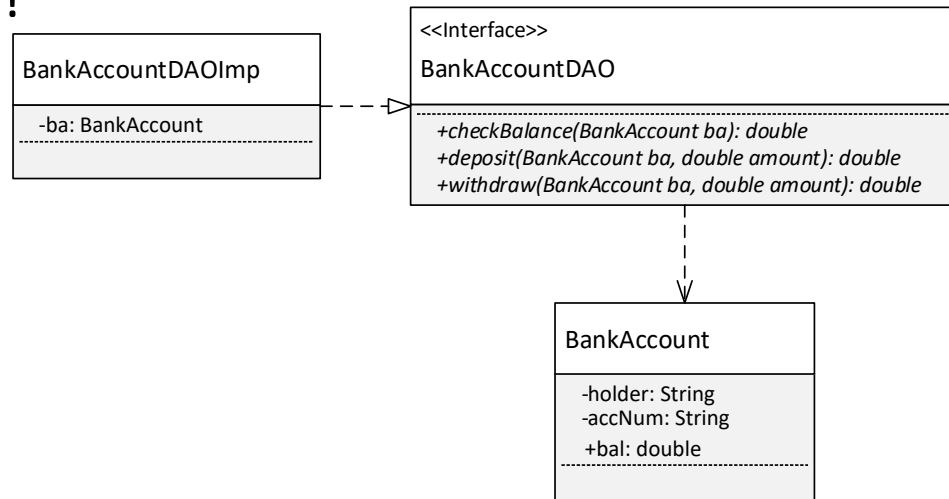


1. **BankAccount** separated from **BankAccountDAO** (the access methods or interface) and the implementation class (**BankAccountDAOImp**)
2. We'll need a Tester class

Example: DAO

❑ The **BankAccount**, **BankAccountDAO** and **BankAccountDAOImp**

❑ Is the diagram the same as above as on previous slide?



1. **BankAccount** separated from **BankAccountDAO** (the access methods or interface) and the implementation class (**BankAccountDAOImp**)
2. We'll need a Tester class

Example: Template pattern

- Self study:
 - What is it used for?
 - Practical example in code

Summary

- Some of the common problems with software development
- Why we use Design patterns
- The main types of Design patterns?
- Examples:
 1. Singleton ✓
 2. Factory ✓
 1. Complete the [Arithmetic, Sum, Subtraction](#) problem
 2. Complete the [Employee, PartTime and FullTime](#) problem
 3. Template method ([self-study](#))
 4. Data Access Object ✓

Questions and comments

?

Next.....

- Data Mapping